

Convocatòria d'Administracions Públiques

VIBE CODING EDUCATIU AMB INTEL·LIGÈNCIA ARTIFICIAL I LA SEUA PUBLICACIÓ

2025-2026

Del 18 juny al 16 de juliol

A distància

32 Hores



GENERALITAT VALENCIANA
Conselleria d'Educació, Cultura i Universitats

CFR | CEFIRE
INTEL·LIGÈNCIA ARTIFICIAL I PENSAMENT COMPUTACIONAL

Índice

¿Qué es el Vibe Coding Educativo? 2
La Filosofía: Programar con "Buena Vibra" . . . 2
Los 4 Rasgos Principales del Vibe Coding . . . 2
Vibe Coding con IA . . . 2
El espectro del vibe coding: de cero código al código asistido . . . 2
¿Qué NO es Vibe Coding? . . . 3
El Flujo de Trabajo en Este Curso . . . 3
Cómo usar este bloque por niveles . . . 3
Herramientas necesarias . . . 3
Mapa general de herramientas y conceptos . . . 3
Configuración del entorno . . . 4
Copilot para docentes . . . 4
Si la licencia educativa GitHub falla (resolución de problemas) . . . 5
¿Por qué necesito Git? . . . 6
Instalación de Git y Configuración de GitHub . . . 6
Guía básica de Git para repositorios GitHub . . . 7
GitHub básico . . . 7
Cómo deshacer cambios . . . 8
Copilot / Chat en VS Code (uso practico) . . . 8
Cline: agente de IA en VS Code (alternativa a Copilot) . . . 9
Manual Básico de Visual Studio Code (aplicable a VSCodium) . . . 10
Buenas prácticas iniciales . . . 11
FAQ de taller (problemas tipicos) . . . 14
Errores al subir a GitHub (push) . . . 14
Módulo 1: Crear el proyecto (versión mínima) . . . 15

¿Qué es el Vibe Coding Educativo?

La Filosofía: Programar con "Buena Vibra"

El término `vibe coding` nace de la idea de programar con una "buena vibra": sin estrés, sin complejidad innecesaria y con un foco absoluto en la utilidad y la satisfacción de crear algo que funciona.

Aplicado a la educación, es una metodología de trabajo que permite a los docentes, sin experiencia previa en programación, construir sus propias herramientas digitales de manera ágil y motivadora. Se trata de resolver una necesidad concreta del aula con una solución simple, directa y que se pueda construir en poco tiempo.

Los 4 Rasgos Principales del Vibe Coding

1. **Objetivos Claros y Acotados:** No intentamos construir la siguiente gran plataforma educativa. En su lugar, nos centramos en un problema muy específico. *Ejemplo: "Necesito una forma rápida para que mi alumnado repase las capitales de Europa", en lugar de "Necesito un sistema de gestión del aprendizaje".*

2. **Entregas Pequeñas y Frecuentes:** Avanzamos con cambios mínimos pero funcionales. Cada pequeño paso resulta en algo que se puede ver y probar. Esto mantiene la motivación alta y permite corregir el rumbo fácilmente. *En lugar de pasar meses planificando, creamos una primera versión en una tarde.*

3. **Revisión Constante con Criterios Didácticos:** La pregunta que guía el proceso no es "¿es tecnológicamente perfecto?", sino "¿es útil para mi alumnado?". La tecnología es un medio, no un fin.

4. **Documentación Mínima pero Útil:** No escribimos manuales extensos. Dejamos notas claras en el código y un archivo `README.md` que explica qué hace la herramienta y cómo usarla. Lo justo para que otro docente (o nuestro "yo" del futuro) pueda entenderlo.

Vibe Coding con IA

En este curso usamos la IA como un copiloto: ayuda a desbloquear ideas, explicar errores y proponer fragmentos de código, pero el criterio didáctico sigue siendo del docente. La IA no sustituye la comprensión del problema; acelera el proceso.

Claves para usar la IA con criterio - Pedir cambios pequeños y comprobables. - Dar contexto (archivo, objetivo, restricciones). - Verificar en el navegador y en la consola. - Documentar lo que se ha hecho para poder repetirlo.

El espectro del vibe coding: de cero código al código asistido

El vibe coding no es un único punto fijo. Es más útil pensarlo como un espectro que va desde el extremo sin código hasta la programación tradicional, y este curso ocupa una posición concreta en ese espectro.

Enfoque	Cómo funciona	Para qué sirve	Ejemplo
Cero código	Describes la app y la IA la despliega directamente	Prototipo rápido, idea de 10 minutos	Actividad de arranque con Copilot
Vibe Coding asistido ← *este curso*	La IA genera el código, tú lo revisas y publicas con tus herramientas	Recursos educativos reales, iterables y compartibles	HTML + VS Code + GitHub Pages
Programación con apoyo de IA	Escribes código y usas la IA para sugerencias y depuración	Proyectos medianos con control total	Desarrollo profesional con Copilot en VS Code
Programación tradicional	Escribes todo el código sin IA	Sistemas complejos, alta seguridad	Ingeniería de software

¿Por qué este curso no se queda en el extremo de cero código?

El enfoque de cero código es potente para empezar, pero tiene límites claros: el resultado es un archivo suelto sin estructura, no puedes guardar versiones anteriores, no puedes publicar con una URL permanente

y si la app crece un poco se vuelve difícil de mantener.

El vibe coding asistido, que es el que practicamos aquí, mantiene la misma idea de describir en lenguaje natural lo que quieres, pero añade un entorno de trabajo (VS Code, Git, GitHub) que hace el resultado sostenible, compartible y publicable. No necesitas aprender a programar, pero sí a entender lo que la IA genera para poder revisarlo y mejorarlo.

¿Qué NO es Vibe Coding?

- **No es un framework o una tecnología:** Es una forma de pensar y trabajar. Se puede aplicar con HTML, con Python o con cualquier herramienta que permita crear rápidamente.
- **No es para proyectos a gran escala:** No lo usarías para construir el software de gestión de un centro escolar. Su fuerza reside en las pequeñas utilidades.
- **No es sinónimo de "código desordenado":** Aunque priorizamos la velocidad y la simpleza, seguimos buenas prácticas para que el código sea legible y mantenible.
- **No es depender de herramientas complejas:** Evitamos las configuraciones enrevesadas. Usamos herramientas accesibles como Visual Studio Code (o VSCodium) y GitHub Pages que simplifican el proceso.

El Flujo de Trabajo en Este Curso

El curso está diseñado para aplicar esta filosofía en un proyecto real:

1. **Módulo 1: Idear.** Definimos un problema (un cuestionario) y su alcance mínimo.
2. **Módulo 2: Construir.** Creamos una primera versión funcional con HTML.
3. **Módulo 3: Mejorar.** Añadimos interactividad y estilos con CSS y JavaScript.
4. **Módulo 4: Publicar y Evaluar.** Dejamos la herramienta lista para usar y la documentamos.

Al final, no solo tendrás una herramienta útil, sino un método de trabajo que podrás aplicar a futuras ideas. # Herramientas, entorno y Git

Cómo usar este bloque por niveles

Nivel básico

Quédate con esto: - VS Code o VSCodium. - Navegador web. - Abrir carpeta, crear archivo, guardar y recargar.

Nivel medio

Añade: - Git como historial de cambios. - GitHub como copia online. - Separación básica del proyecto en carpetas.

Nivel avanzado

Explora además: - terminal integrada - ramas - `push`, `pull`, remotos - publicación y trabajo más autónomo

Herramientas necesarias

Antes de instalar, es importante saber qué es cada herramienta y para qué sirve:

- **Visual Studio Code (VS Code) o VSCodium:** es el programa donde vas a escribir y guardar el código (como un cuaderno digital). VSCodium es la alternativa libre.
- **Navegador web (Chrome o Firefox):** es donde vas a ver lo que has creado, como si fuera el "visor" de tu proyecto.
- **Git:** es un sistema que guarda el historial de cambios, como si fueran "versiones" de tu trabajo.
- **GitHub:** es una página web donde puedes guardar tu proyecto en la nube y publicarlo.
- **Correo `nombredeusuario@edu.gva.es`:** se usa para identificarte en GitHub como parte de la comunidad educativa. Si ya tienes cuenta en GitHub, este paso es opcional.

Mapa general de herramientas y conceptos

Antes de entrar en instalaciones y botones, conviene tener una "vista de mapa" del ecosistema:

Pieza	Para que sirve
GitHub	Alojar repositorios y colaborar. Para este curso, piensa en GitHub como una "carpeta en la nube" con historial y opción de publicar (GitHub Pages).
Git	Control de versiones: guarda el historial de cambios ("fotos" del proyecto).
VS Code	Editor/IDE local para trabajar con archivos de texto (HTML, CSS, JS, Markdown…).
Copilot / Chat (o Gemini)	Capa de IA dentro del editor: preguntas, propuestas de cambios, generación de archivos, ayuda para depurar.
Modelos	"Motores" de IA (según proveedor). Cada uno tiene límites, costes y sesgos.
Tokens	Presupuesto de uso del modelo. En general, trabajar con texto plano consume menos que trabajar con PDFs o imágenes.
Markdown	Formato de texto plano ideal para apuntes y documentación. Fácil de versionar con Git.
Python	Útil para generar figuras y gráficas por código (repetible y modificable).

Ecosistema y licencias (nota rapida)

- GitHub y VS Code pertenecen al ecosistema Microsoft, y por eso se integran muy bien entre si.
- GitHub Copilot suele estar mejor integrado en VS Code, pero no es imprescindible para hacer el curso.
- La licencia educativa de GitHub (si aplica) puede habilitar mejores capacidades, pero puedes trabajar con alternativas gratuitas.
- Recomendación práctica: filtra tutoriales por "último año", porque estas herramientas cambian rápido.

Alternativas y herramientas complementarias (opcional)

No es necesario usarlas en este curso, pero conviene saber que existen:

- **Canvas (ChatGPT / Gemini / Claude):** para crear contenido o una web desde el chat y compartirla rapido por URL (menos control que VS Code).
- **Cursor:** IDE con IA muy potente (normalmente de pago).
- **Perplexity:** útil para búsquedas con fuentes (sus planes cambian con frecuencia).
- **NotebookLM:** muy útil para trabajar con muchos documentos, resumir y convertir a Markdown (por ejemplo, para normativa).

Configuración del entorno

Ruta básica recomendada

1. Instala Visual Studio Code o VSCodium.
2. Crea una carpeta para tu primer proyecto y ábrela en VS Code con `Archivo > Abrir carpeta...`.
3. Crea `index.html`, guarda y ábrelo en el navegador.

Ruta media y avanzada

4. Instala Git.
5. Crea o configura tu cuenta de GitHub y asocia el correo `@edu.gva.es`.

Copilot para docentes

Esta parte queda como apoyo especialmente útil para niveles medio y avanzado.

Paso 1 · Tener cuenta en GitHub

Si ya tienes cuenta, perfecto. Si no:

Ve a https://github.com

Crea una cuenta (puede ser con tu correo personal).

Añade después tu correo docente @edu.gva.es en Settings → Emails (esto ayuda mucho en la verificación).

Paso 2 · Solicitar acceso como docente (GitHub Education)

Entra aquí: https://education.github.com/teachers

Pulsa "Get benefits".

Identifícate con tu cuenta de GitHub.

Elige Teacher.

Verificación:

Normalmente basta con el correo institucional @edu.gva.es

Si pide documento, sirve un certificado de destino, nombramiento, o captura de ITACA (sin datos sensibles).

El proceso suele tardar desde minutos hasta 1–2 días.

Cuando te aprueban, ya tienes acceso educativo completo.

Paso 3 · Activar GitHub Copilot (Pro)

Una vez aprobado:

Ve a: https://github.com/settings/copilot

Activa Copilot.

Verás que tienes Copilot Pro sin coste mientras seas docente verificado.

Si la licencia educativa GitHub falla (resolución de problemas)

La solicitud de los pasos anteriores puede fallar por detalles pequeños. Si te ocurre, no bloquees el curso: puedes seguir con el plan gratuito o con otros modelos/asistentes y volver a intentarlo más tarde.

Esta sección resume lo que suele funcionar en la práctica y los fallos más comunes.

> **Recordatorio de requisitos:** cuenta de GitHub creada, correo institucional/educativo añadido y verificado, y la solicitud del beneficio enviada (los tres pasos anteriores). Algunos correos personales (por ejemplo, Gmail) no sirven como evidencia de docencia; si tienes correo institucional, añádelo aunque mantengas el personal como principal.

Errores típicos observados

- Denegaciones repetidas aunque la persona sea docente.
- Verificación sensible a:
  - calidad de cámara o iluminación
  - documento usado
  - coincidencia exacta del nombre

Checklist de verificación (antes de reintentar)

Cuenta y correos - Ve a `Settings -> Emails`. - Añade el correo institucional si lo tienes y verifica TODOS los correos que aparezcan como pendientes. - Ten claro cuál es el correo primario (no siempre importa, pero evita confusiones).

Nombre coherente - Revisa el nombre en el perfil público: usa tu nombre real y completo. - Si en alguna parte de la cuenta te pide nombre para facturación/perfil, usa exactamente el mismo (mismo orden y tildes, sin abreviaturas).

Documento y cámara - Si el proceso usa verificación visual, prueba con mejor cámara (móvil suele ir mejor que una webcam floja). - Evita reflejos, sombras y baja resolución. - Si tienes varios documentos válidos, prueba con otro.

Tiempos - La respuesta puede tardar desde minutos hasta varios días. No asumas que "si no sale al instante, ha fallado".

Reintentos - Tras un fallo, a veces el sistema se vuelve más exigente. Antes de reintentar, corrige una sola cosa cada vez (correo, nombre, cámara) para aislar el problema.

Estrategias si no funciona (para no bloquear el curso)

- Empieza igualmente con el plan gratuito: para aprender VS Code, Markdown, Git y el flujo del curso es suficiente.
- Alterna herramientas según tarea:
  - búsqueda y verificación rápida: buscadores con fuentes o documentación oficial
  - redacción de documentación: texto plano en Markdown
- Apóyate en la comunidad/compañeros: ver 2-3 casos reales ayuda a detectar el "detalle tonto" que está bloqueando.

¿Por qué necesito Git?

Imagina que estás corrigiendo una unidad didáctica en Word. A mitad del trabajo te das cuenta de que la versión de hace tres días era mejor. ¿La tienes guardada? ¿Sabes cuál era exactamente?

Git resuelve ese problema: guarda una foto del proyecto cada vez que tú lo indicas (un *commit*), con un mensaje que explica qué cambiaste. Puedes volver a cualquier foto anterior cuando quieras.

Para el nivel básico de este curso, **Git es opcional**: puedes crear y publicar tu recurso sin tocarlo. Pero si alguna vez has borrado algo que no querías borrar, o has "mejorado" algo que luego quedó peor, ya entiendes por qué existe.

Tres situaciones en las que Git te salva:

- Probaste un cambio grande en el CSS y ahora la página está rota. Con Git, vuelves al estado anterior en segundos.
- Has trabajado en el mismo proyecto desde el ordenador del aula y desde casa. Git mantiene todo sincronizado.
- Un compañero quiere usar tu recurso como base para el suyo. Le compartes el repositorio y cada uno trabaja en su copia.

Si estás en nivel básico y todo esto te suena lejano, sáltate la instalación por ahora y vuelve cuando lo necesites.

Instalación de Git y Configuración de GitHub

Esta parte es especialmente útil para niveles medio y avanzado. Si estás en nivel básico, puedes dejarla para más adelante.

> **Ampliación recomendada:** si quieres profundizar paso a paso en Git, tienes disponible el curso de Aules Curso sobre Git.

1. Crear una cuenta en GitHub

- Antes de nada, si no tienes una, crea una cuenta en GitHub. Es gratuita y será donde alojes tus proyectos.
- Asocia tu correo `@edu.gva.es` a tu cuenta de GitHub para identificarte como miembro de la comunidad educativa.

2. Instalar Git

- **En Windows:** Descarga e instala Git for Windows. Ejecuta el instalador y sigue los pasos. Puedes dejar las opciones por defecto, pero asegúrate de que la opción "Git from the command line and also from 3rd-party software" esté seleccionada.
- **En Mac:** La forma más sencilla es instalar las "Command Line Tools" de Xcode. Abre la terminal (puedes buscarla con Spotlight) y escribe `git --version`. Si no lo tienes, el sistema te pedirá instalarlo.
- **En Linux:** Abre una terminal y usa el gestor de paquetes de tu distribución.
  - En Debian/Ubuntu: `sudo apt update && sudo apt install git`
  - En Fedora: `sudo dnf install git`

3. Configurar Git por primera vez

- Una vez instalado, abre una terminal (o Git Bash en Windows) y configura tu nombre de usuario y correo. Estos datos se asociarán a tus `commits`.

- ```
  git config --global user.name "Tu Nombre"
  git config --global user.email "tu_correo@edu.gva.es"
  ```

- Reemplaza `"Tu Nombre"` y `"tu_correo@edu.gva.es"` con tus datos reales.

Guía básica de Git para repositorios GitHub

Conceptos clave

- **Repositorio:** carpeta del proyecto con historial.
- **Commit:** instantánea de los cambios con un mensaje.
- **Rama (`main`):** línea principal de trabajo.
- **Remoto (`origin`):** copia del repositorio en GitHub.

Nota: no es necesario hacer todo este proceso desde la terminal. En VS Code puedes ver los cambios, hacer `commit` y `push` desde el panel de Control de código fuente.

GitHub básico

Este apartado resume lo esencial de GitHub para empezar con buen pie, sin entrar en demasiados detalles.

Qué es Git y qué es GitHub

- **Git** guarda el historial de cambios de tu proyecto.
- **GitHub** es el lugar online donde subes tu proyecto para compartirlo y publicarlo.

Flujo sencillo en GitHub

- **Repositorio:** es la carpeta del proyecto en GitHub.
- **Clonar:** descargar el repositorio a tu ordenador para trabajar con él.
- **Commit:** guardar un "punto de control" con un mensaje claro.
- **Push:** subir esos cambios a GitHub.

Términos que verás a menudo

- **README:** archivo que explica qué es el proyecto y cómo se usa.
- **Branch (rama):** copia del proyecto para probar cambios sin romper lo principal.
- **Pull request:** forma de proponer cambios y pedir revisión antes de unirlos.
- **Issue:** lista de tareas o problemas pendientes.

Perfil y comunidad

- Tu perfil muestra tus proyectos y tu actividad.
- Puedes "marcar con estrella" repositorios para guardarlos.
- Explora proyectos en `github.com/explore` si quieres inspiración.

Siguiente paso (opcional)

- Crea un repositorio nuevo y practica: añade un README, haz un commit y publícalo.

Flujo básico de trabajo

1. Antes de empezar, actualiza el repositorio: `git pull` (trae lo último de GitHub).
2. Revisa el estado de los archivos: `git status` (muestra qué ha cambiado).
3. Añade los cambios: `git add .` (prepara los archivos para guardar).
4. Crea un commit: `git commit -m "Mensaje claro"` (guarda una versión).
5. Envía los cambios a GitHub: `git push` (sube la versión a la nube).

Si te resulta más fácil, puedes hacer estos pasos desde el panel de Git de VS Code, sin usar la terminal.

> **[IA] Cómo pedirle a la IA**
>
> "Haz git add de todos los archivos modificados, escribe un mensaje de commit claro que resuma lo que he cambiado y ejecuta el push a GitHub."

Crear o conectar un repositorio

- **Clonar un repositorio existente:** `git clone URL_DEL_REPO`
  ```

- **Crear un repositorio nuevo:**
  - Créalo en GitHub y copia la URL.
  - En la carpeta local: `git init`
  - Añade el remoto: `git remote add origin URL_DEL_REPO`
  - Primer envío: `git push -u origin main`

Cómo deshacer cambios

Cometer errores es parte del proceso. Git y VS Code ofrecen varias formas de volver atrás según en qué punto te encuentres.

Si aún no has guardado (antes de Ctrl+S)

`Ctrl+Z` (Windows/Linux) o `Cmd+Z` (Mac) deshace los últimos cambios en el archivo abierto, igual que en cualquier procesador de texto.

Si has guardado pero no has hecho commit

El archivo está guardado pero Git aún no lo ha "fotografiado". Puedes descartar los cambios y volver al último commit:

- **Desde VS Code:** en el panel de Control de código fuente, haz clic derecho sobre el archivo modificado y elige *Descartar cambios*.
- **Desde la terminal:** `git restore nombre-del-archivo`

  Cuidado: esta acción no se puede deshacer con Ctrl+Z. El archivo vuelve al último commit y los cambios desaparecen.

Si ya has hecho commit pero el resultado no te convence

Has guardado una versión con `git commit` y quieres volver atrás sin perder el historial:

- **Desde VS Code:** abre la Paleta de comandos (`Ctrl+Shift+P`), escribe *Git: Undo Last Commit*. Deshace el último commit pero mantiene los cambios en los archivos para que puedas editarlos.
- **Desde la terminal:** `git revert HEAD` crea un nuevo commit que deshace el anterior. Es el método más seguro porque no reescribe el historial.

Regla general

Haz commits pequeños y frecuentes. Cuanto más pequeño es el commit, más fácil es deshacer exactamente lo que no funcionó sin perder el resto del trabajo.

> **[IA] Cómo pedirle a la IA**
>
> "El último cambio ha roto algo. Revierte el último commit sin reescribir el historial y deja el proyecto en el estado anterior."

Copilot / Chat en VS Code (uso practico)

Esta sección es un "mini manual" de lo que más se usa en un taller real: iniciar sesión, ver que está bien conectado y evitar perder trabajo.

Autenticación

- Si VS Code te lo pide, elige "Continue with GitHub" (o el proveedor correspondiente) y autoriza en el navegador.
- Vuelve a VS Code y espera a que termine la conexión.

Senales de que esta listo: - Ves el panel/chat del asistente disponible. - Puedes escribir una pregunta simple sobre un archivo abierto y responde.

Sincronización (qué se sincroniza y qué no)

VS Code puede sincronizar configuración (tema, extensiones, ajustes). El historial del chat no siempre se sincroniza de forma fiable entre equipos.

Regla práctica: lo importante se guarda en texto. - Si una conversación contiene decisiones, prompts útiles o criterios, cópiala a un `.md` del proyecto. - Si el asistente te propone un plan bueno, guárdalo como checklist

en el `README.md`.

Menciones útiles en el chat

Según el asistente, puedes tener menciones como: - `@workspace`: preguntas sobre tu proyecto (archivos, estructura). - `@vscode`: dudas sobre el IDE (atajos, opciones, como hacer algo).

Modelos y API keys (idea didáctica)

Algunas integraciones funcionan sin que tú aportes nada más. Otras piden una API key.

Criterio para el curso: - Si te pide API key y no la tienes, no es un fallo: sigue con el flujo del curso con lo que tengas disponible. - Prioriza aprender el método (planificar, probar, iterar, documentar) por encima de "tener el modelo más potente".

Cline: agente de IA en VS Code (alternativa a Copilot)

> **Nivel medio y avanzado.** Cline es una extensión de VS Code que funciona como **agente autónomo**: a diferencia de Copilot (que te sugiere código mientras escribes), Cline puede leer tus archivos, modificarlos y ejecutar comandos en el terminal, siempre pidiéndote permiso antes de aplicar cambios. Es una opción complementaria a GitHub Copilot, especialmente útil cuando trabajas con proyectos de varios archivos o quieres una asistencia más autónoma.

Qué necesitas

- VS Code instalado (el mismo editor que ya estás usando).
- Una cuenta de Cline para autenticar la extensión.

Paso 1 · Instalar la extensión Cline

1. Abre VS Code.
2. Ve al panel **Extensiones** (o pulsa `Ctrl+Shift+X`).
3. Busca **"Cline"** (desarrollada por *saoudrizwan*).
4. Pulsa **Instalar**.

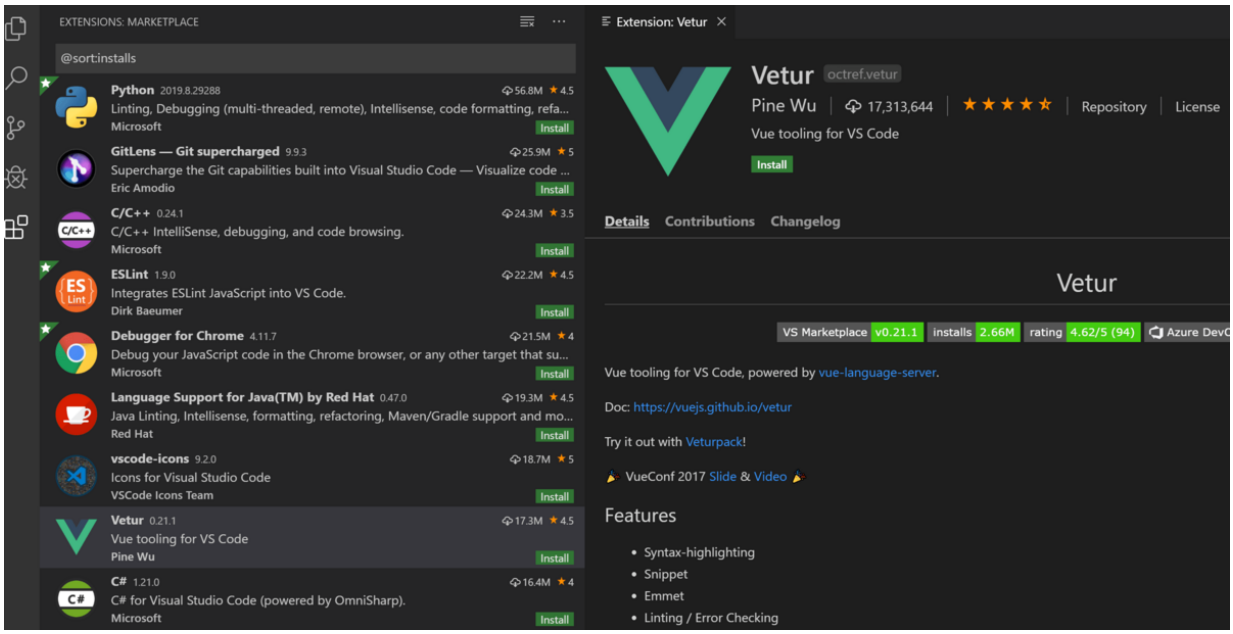


Figura 1: Panel de Extensiones de VS Code, donde buscas la extensión y la instalas con el botón Install

Una vez instalada, verás el icono de Cline en la barra lateral izquierda.

Paso 2 · Registrarse y autenticarse en Cline

Cline requiere una cuenta para funcionar. Tienes dos opciones:

Opción A: crear una cuenta nueva en Cline 1. Abre el panel de Cline (icono en la barra lateral).

2. Cuando te pida iniciar sesión, selecciona **"Sign up"** o **"Create account"**. 3. Introduce tu correo electrónico y crea una contraseña. 4. Verifica el correo que recibirás para confirmar la cuenta. 5. Vuelve a VS Code e inicia sesión con las credenciales creadas.

Opción B: utilizar una cuenta existente (GitHub, Google, etc.) 1. Abre el panel de Cline. 2. Selecciona **"Sign in with GitHub"** (u otro proveedor disponible). 3. Autoriza a Cline a acceder a tu cuenta de GitHub. 4. Vuelve a VS Code; la extensión detectará la sesión automáticamente.

> **Consejo:** si ya tienes cuenta de GitHub, la opción B es la más rápida. Si prefieres no vincular cuentas, utiliza la opción A.

Paso 3 · Primer uso

1. Abre la carpeta de tu proyecto en VS Code (`Archivo > Abrir carpeta`).
2. Haz clic en el icono de Cline en la barra lateral para abrir el chat.
3. Escribe una instrucción, por ejemplo: *"Crea un archivo index.html con un cuestionario de 3 preguntas sobre fracciones para 1 ESO."*
4. Cline te mostrará las acciones que piensa hacer (qué archivo creará, qué cambios aplicará).
5. **Revisa las acciones** y, si estás de acuerdo, pulsa **"Accept"**. Si quieres modificar algo, pídelo antes de aprobar.

A partir de ahí, puedes iterar igual que con cualquier IA: *"Añade una cuarta pregunta"*, *"Haz los botones más grandes"*, etc.

Cuándo elegir Cline en vez de Copilot

- **Proyectos de varios archivos:** Cline lee el contexto completo del proyecto, no solo el fragmento que tienes abierto.
- **Flujo autónomo:** puede ejecutar comandos (instalar dependencias, lanzar el servidor, leer errores) y corregir sin que tengas que copiar y pegar.
- **Memoria automática:** guarda el contexto entre sesiones sin que tengas que repetir la información.
- **Permisos detallados:** apruebas acción por acción, lo que te da control total sobre los cambios.

Consejos para empezar

- **Comienza con permisos restrictivos.** Aprueba solo las acciones que entiendas. Evita el modo "auto-approve" mientras aprendes.
- **Trabaja con Git.** Antes de una sesión larga, haz un `commit`. Así podrás volver atrás si algo no funciona.
- **Revisa siempre el código.** Cline es muy potente, pero la responsabilidad final del recurso educativo es tuya: revisa el contenido, la accesibilidad y la exactitud pedagógica.
- **No incluyas datos sensibles.** No trabajes con datos personales del alumnado ni documentos confidenciales en el proyecto.

> **Nota:** Cline es una herramienta en evolución constante. Los comandos y la interfaz pueden cambiar; consulta su documentación oficial para la información más actualizada.

Manual Básico de Visual Studio Code (aplicable a VSCodium)

Visual Studio Code (VS Code) es un editor muy potente, pero para este curso nos centraremos en sus funciones más básicas. Si usas VSCodium, encontrarás la misma estructura general y opciones equivalentes.

Interfaz Principal

La interfaz se divide en varias áreas:

1. **Barra Lateral (Activity Bar):** A la izquierda, te da acceso a las vistas principales:
   - **Explorador (icono de archivos):** Para ver y gestionar los archivos y carpetas de tu proyecto.
   - **Búsqueda (icono de lupa):** Para buscar texto en todos tus archivos.
   - **Control de código fuente (icono de Git):** Para gestionar tus cambios con Git (verás los archivos modificados, podrás hacer `commit`, etc.).
   - **Extensiones (icono de cuadrados):** Para buscar e instalar extensiones como Gemini o Copilot.
2. **Editor:** El área principal donde escribes y editas tu código. Puedes tener varios archivos abiertos en diferentes pestañas.

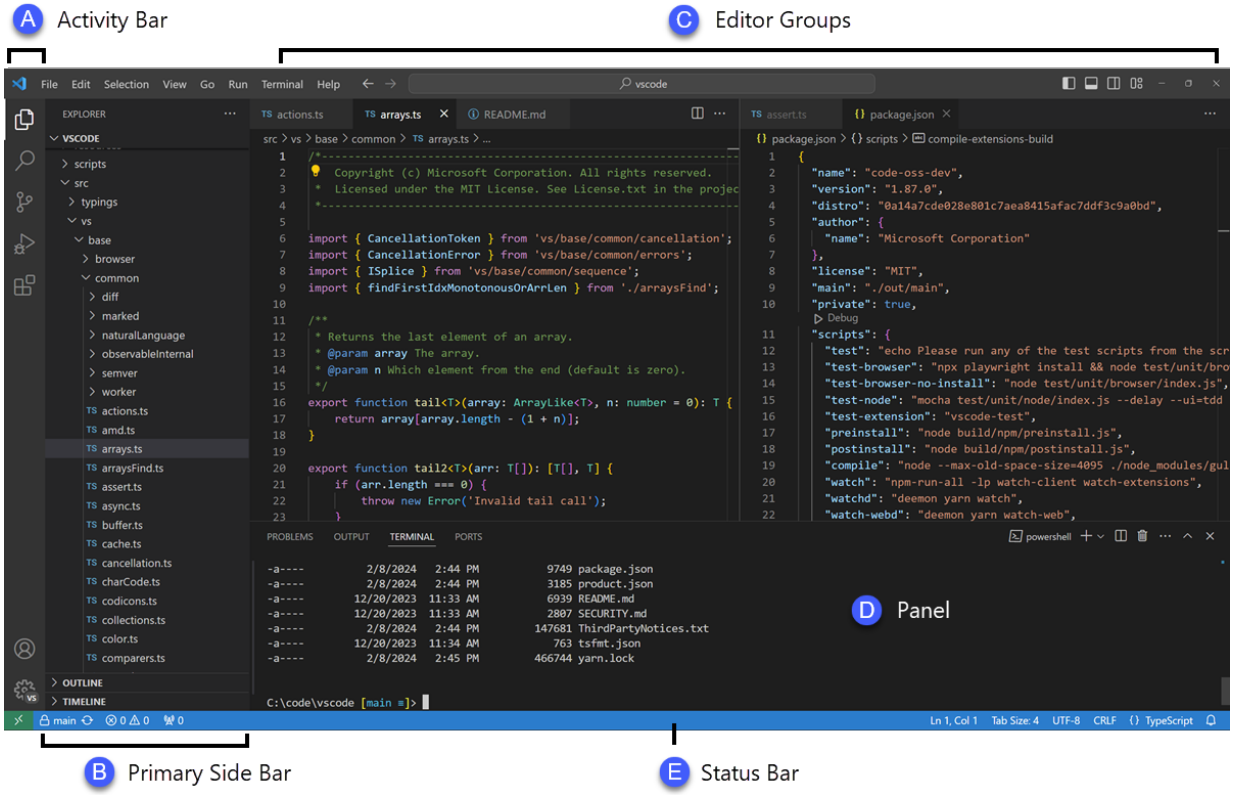


Figura 2: Interfaz principal de VS Code

3. **Panel Inferior (Terminal):** Puedes abrir una terminal integrada (`Ver > Terminal` o `Ctrl+ñ`). Es muy útil para ejecutar comandos de Git sin salir del editor.
4. **Barra de Estado:** La barra azul en la parte inferior. Muestra información útil, como la rama de Git en la que te encuentras.

Capturas de apoyo:

Acciones Esenciales para el Curso

- **Abrir un proyecto:** No abras archivos sueltos. Usa `Archivo > Abrir carpeta...` y selecciona la carpeta principal de tu proyecto. Esto permite a VS Code entender todo el contexto de tu trabajo.
- **Crear un nuevo archivo:** Haz clic derecho en el Explorador de archivos o usa el icono de "nuevo archivo".
- **Guardar cambios:** `Archivo > Guardar` o el atajo `Ctrl+S` (o `Cmd+S` en Mac).
- **Usar la Paleta de Comandos:** Es el centro de operaciones de VS Code. Ábrela con `Ctrl+Shift+P` (o `Cmd+Shift+P` en Mac). Desde aquí puedes buscar y ejecutar casi cualquier comando, como "Git: Commit" o "Git: Push".
- **Interactuar con Gemini o Copilot:** Una vez instalado, el asistente se integra en el editor. Podrás hacerle preguntas sobre tu código, pedirle que genere fragmentos o que te explique errores. Busca los iconos o menús contextuales del asistente mientras programas.

Capturas tomadas de la documentación oficial de VS Code (https://code.visualstudio.com/docs).

Buenas prácticas iniciales

Nivel básico

- Guarda con nombres claros (ej. `index.html`, `estilos.css`).
- Revisa cada cambio en el navegador antes de seguir.

Nivel medio

- Trabaja en pasos pequeños y haz `commits` frecuentes con Git.
- Organiza archivos y carpetas con lógica simple.

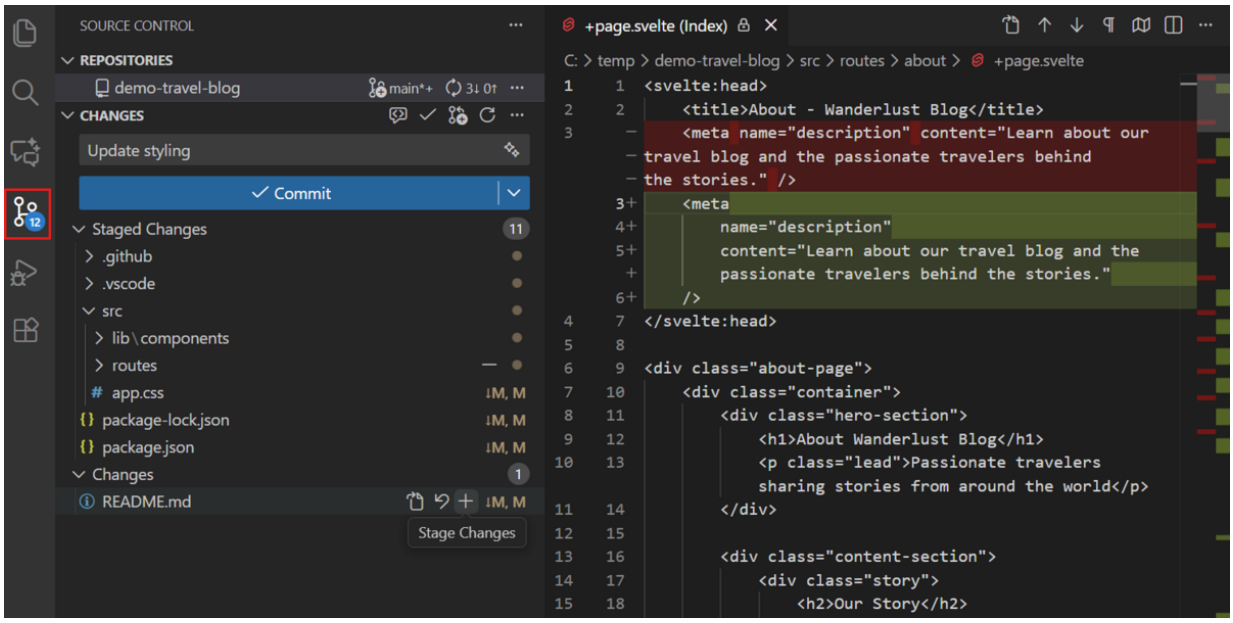


Figura 3: *Control de código fuente*

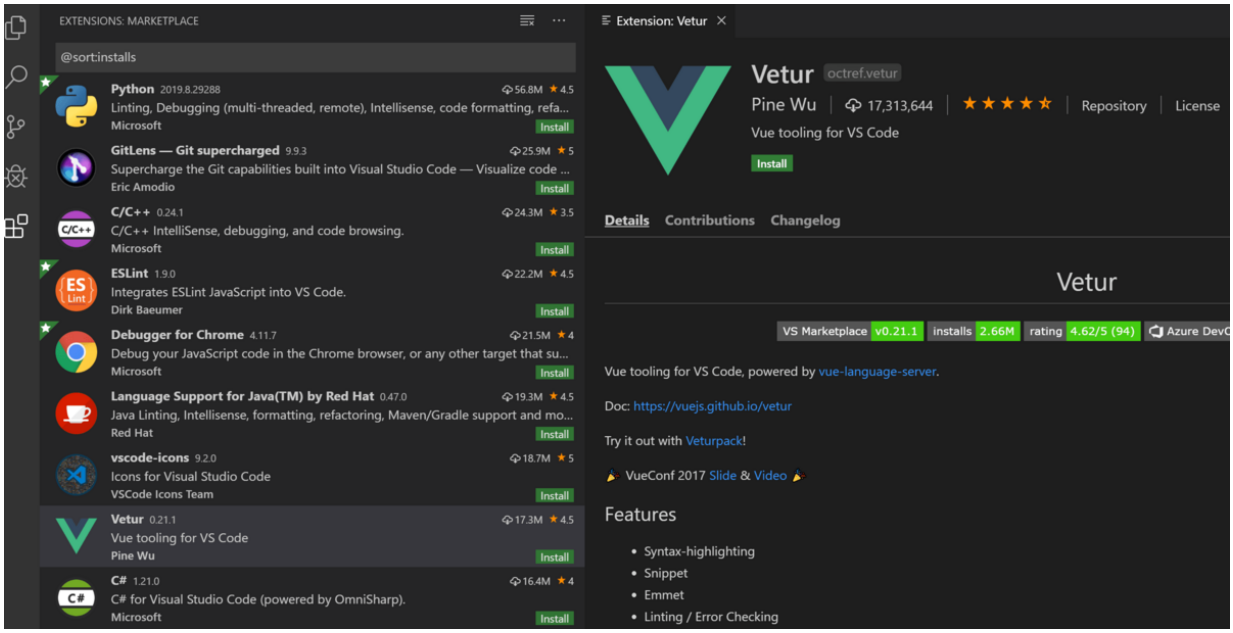


Figura 4: *Extensiones*

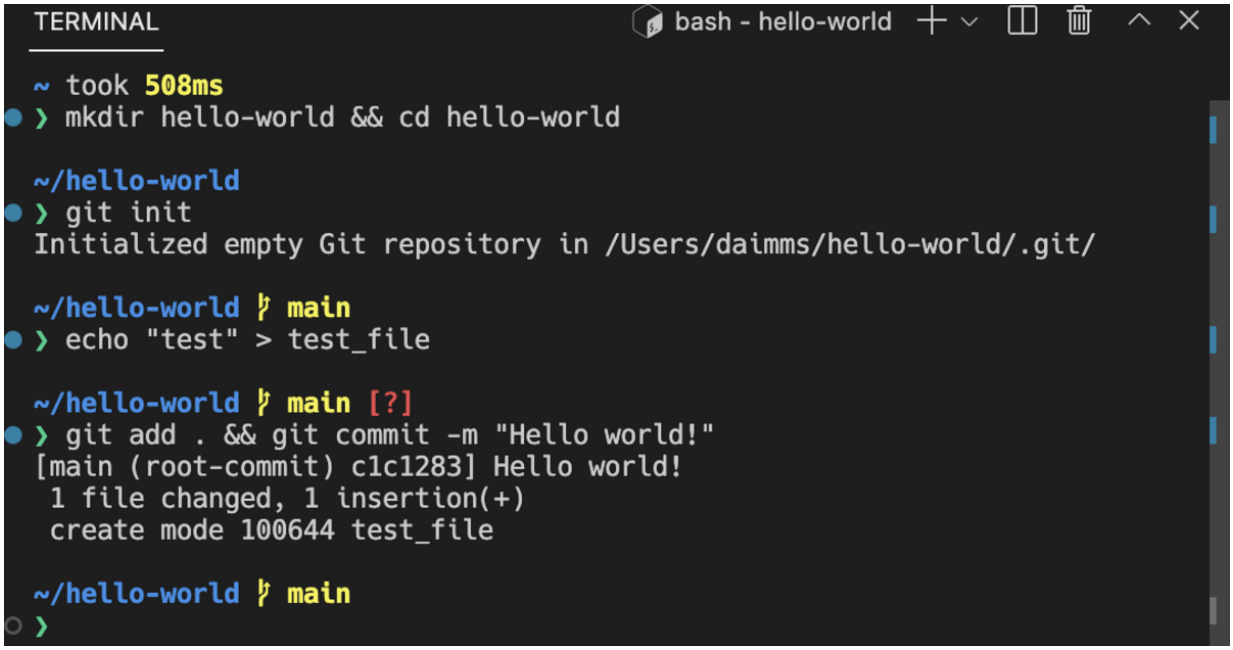


Figura 5: *Terminal integrada*

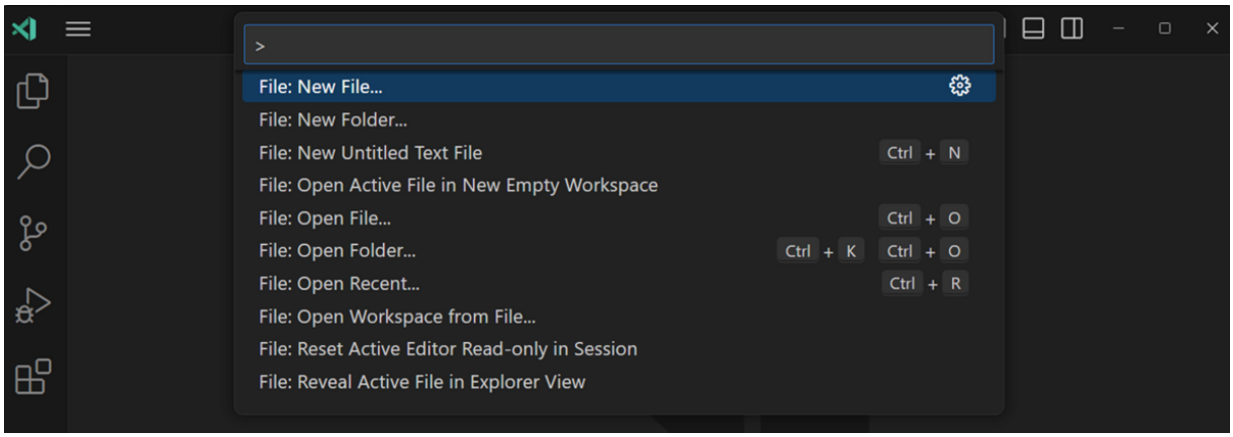


Figura 6: *Paleta de comandos*

Nivel avanzado

- Usa terminal, ramas y flujos más completos cuando realmente te aporten valor.

FAQ de taller (problemas tipicos)

Problema	Solución rápida
"No me aparece el chat/asistente"	Revisa extensiones instaladas, recarga VS Code y comprueba que estás con sesión iniciada.
"Me pide iniciar sesión una y otra vez"	Cierra sesión y vuelve a entrar una sola vez (navegador + VS Code). A veces ayuda reiniciar VS Code.
"He perdido un prompt bueno"	Copia lo importante a un `.md` del proyecto (por ejemplo `notas/ia.md` o el `README.md`).
"La licencia educativa falla"	Revisa la sección de checklist y no bloquees el curso: sigue con el plan gratuito y reintenta más tarde.
"No encuentro un archivo"	Asegúrate de haber abierto una carpeta (no un archivo suelto) en VS Code.
"Me da error al subir a GitHub (push)"	Casi siempre es autenticación o rama. Mira la sección siguiente.

Errores al subir a GitHub (push)

Subir el primer proyecto (`git push`) es donde más se atasca el alumnado. Estos son los tropiezos habituales, del más común al menos, con su solución y un prompt listo para pedírselo a la IA.

1 · "Authentication failed" o te pide usuario y contraseña

GitHub **ya no acepta la contraseña normal** para subir desde la terminal. Dos opciones (basta con una):

- **Opción A — Iniciar sesión desde VS Code (la más fácil):** icono de la **personita** (abajo a la izquierda) → **"Iniciar sesión con GitHub"** → **"Authorize"** en el navegador → vuelve a VS Code y reintenta el push.
- **Opción B — Token personal (si la A no funciona):** en GitHub, *Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic)*, marca la casilla **`repo`**, genera y **copia** el token. Cuando te pida la **contraseña**, pega el **token** (no tu contraseña real).

2 · Se abre el navegador pidiendo permiso

No es un error: es el paso que conecta tu cuenta. Pulsa **"Authorize"** y vuelve a VS Code.

3 · "Permission denied" / "403"

Comprueba que la **URL del repo es la tuya** (tu usuario en la dirección). Si copiaste la de un compañero o la del ejemplo, no tendrás permiso: copia la URL desde el botón verde **"Code"** de tu propio repositorio.

4 · "master vs main" o "src refspec main does not match"

La rama local se llama distinto que en GitHub. Prompt para la IA:

> Renombra la rama local a "main" y haz push de la rama main al repositorio remoto, estableciéndola como rama por defecto.

(Equivale a `git branch -M main` y `git push -u origin main`.)

5 · "Updates were rejected" / "failed to push some refs"

En GitHub ya hay algo (p. ej. marcaste *"Add a README"* al crear el repo) que tu copia local no tiene. Prompt para la IA:

> Trae primero los cambios del repositorio remoto y combínalos con los míos, resolviendo el historial, y luego sube mis cambios.

(Equivale a `git pull --rebase` y después `git push`.)

Si nada de esto lo desbloquea, pide una **captura del mensaje de error completo**: casi siempre indica cuál de estos cinco casos es.

Módulo 1: Crear el proyecto (versión mínima)

El objetivo de este módulo es tener algo visible en el navegador, aunque sea muy simple. No importa que no funcione del todo: lo mejorarás en los módulos siguientes.

- **Imprescindible:** completar el ejercicio de nivel básico.
- **Opcional:** avanzar en nivel medio o avanzado si ya tienes la base estable.
- **Tiempo orientativo:** 30-45 min (básico) + 20-30 min por ampliación.

Nivel básico

Ejercicio 1: crear `index.html` con una pregunta **Qué vas a hacer:** crear un archivo HTML con un título, una pregunta de tipo test y cuatro posibles respuestas.

Pasos:

1. Crea una carpeta nueva para el proyecto. Ejemplo: `cuestionario-capitales`.
2. Abre esa carpeta en VS Code con `Archivo > Abrir carpeta...`.
3. Crea un archivo nuevo llamado `index.html` (botón derecho en el panel de archivos > `Nuevo archivo`).
4. Escribe el contenido inicial o pídele a la IA que lo genere.

Prompt sugerido:

```
Crea un archivo index.html con:
- Título de la página: "Cuestionario de [tu tema]"
- Un encabezado h1 con el nombre del cuestionario
- Una pregunta de tipo test con 4 respuestas posibles (radio buttons)
- Un botón con el texto "Corregir"
Usa HTML correcto y sencillo, sin CSS externo ni JavaScript todavía.
```

5. Guarda el archivo (`Ctrl+S`).
6. Abre el archivo en el navegador: haz clic derecho sobre `index.html` en VS Code y elige `Reveal in File Explorer` (o `Mostrar en el explorador`), luego abre el archivo desde ahí haciendo doble clic.

Resultado esperado: verás una página en blanco con texto, una pregunta con sus opciones y un botón. Sin estilos, sin colores, sin funciones: eso es correcto para esta etapa.

Checklist de fin de etapa

☐ El archivo `index.html` existe en la carpeta del proyecto.
☐ Se abre en el navegador sin mostrar errores.
☐ Se ve el título, la pregunta y las respuestas.
☐ Hay un botón "Corregir" visible.

Nivel medio

Ejercicio 2: iniciar el repositorio y hacer el primer commit **Qué vas a hacer:** conectar tu carpeta de proyecto a Git y subir los archivos a GitHub.

Pasos:

1. Asegúrate de tener Git instalado. En la terminal de VS Code (`Ver > Terminal`), escribe `git --version`. Si aparece un número de versión, Git está instalado.
2. Acepta el enlace de GitHub Classroom si te lo han facilitado, o crea un repositorio nuevo en GitHub:
   - Ve a github.com, inicia sesión y pulsa el botón `+` (arriba a la derecha) > `New repository`.
   - Dale un nombre, déjalo público y pulsa `Create repository`.
3. Pídele a la IA que haga el primer commit y lo suba:

Prompt sugerido:

```
Inicia un repositorio Git en esta carpeta, haz el primer commit con todos los archivos
y súbelos al repositorio de GitHub con esta URL: [pega aquí la URL del repo].
```

4. Comprueba en GitHub que los archivos han aparecido.

Checklist de fin de etapa

☐ El repositorio existe en GitHub con el `index.html` subido.
☐ El commit tiene un mensaje descriptivo (no "asdf" o "cambios").
☐ Has añadido un `README.md` con al menos una frase explicando el proyecto.

Nivel avanzado

Ejercicio 3: activar GitHub Pages **Qué vas a hacer:** publicar tu proyecto para que cualquier persona pueda verlo con una URL.

Pasos:

1. Entra al repositorio en GitHub.
2. Ve a `Settings` (pestaña de configuración, arriba a la derecha del repositorio).
3. En el menú lateral, busca `Pages`.
4. En `Build and deployment`, selecciona:
   - Source: `Deploy from a branch`
   - Branch: `main` y carpeta `/ (root)`
5. Pulsa `Save`.
6. Espera 1-2 minutos y recarga la página. Aparecerá una URL del tipo `https://tu-usuario.github.io/nombre-repo/`.
7. Abre esa URL y comprueba que se ve el `index.html`.

Prompt sugerido si algo no funciona:

```
GitHub Pages no carga mi proyecto. El repositorio es [URL].
Revisa la configuración y dime qué está mal.
```

Checklist de fin de etapa

☐ La URL de GitHub Pages carga correctamente.
☐ El proyecto se ve igual que en local.
